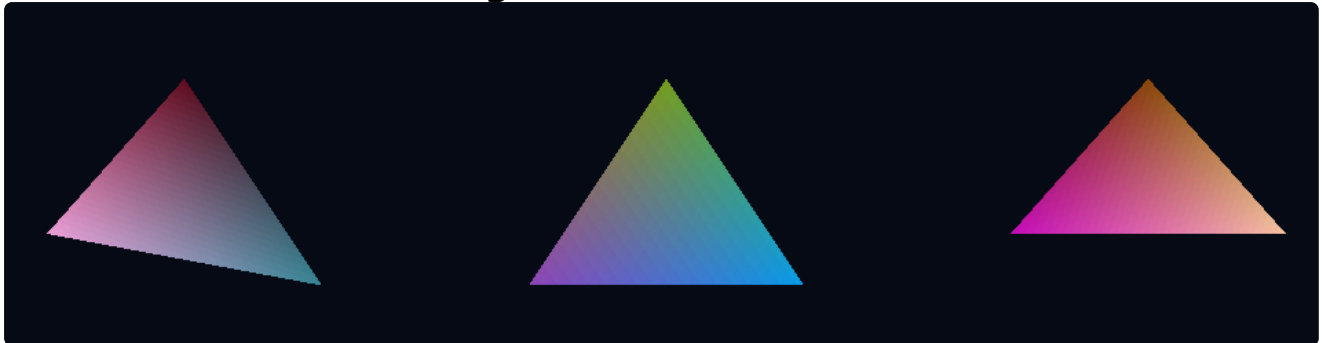


Assignment 2 TDT4195 Fall26

Task 1: Per-Vertex Colors

1b.) Render a scene containing at least 3 different triangles, where each vertex of each triangle has a different color



Created the method `generate_colors` to create random colors for each vertex.

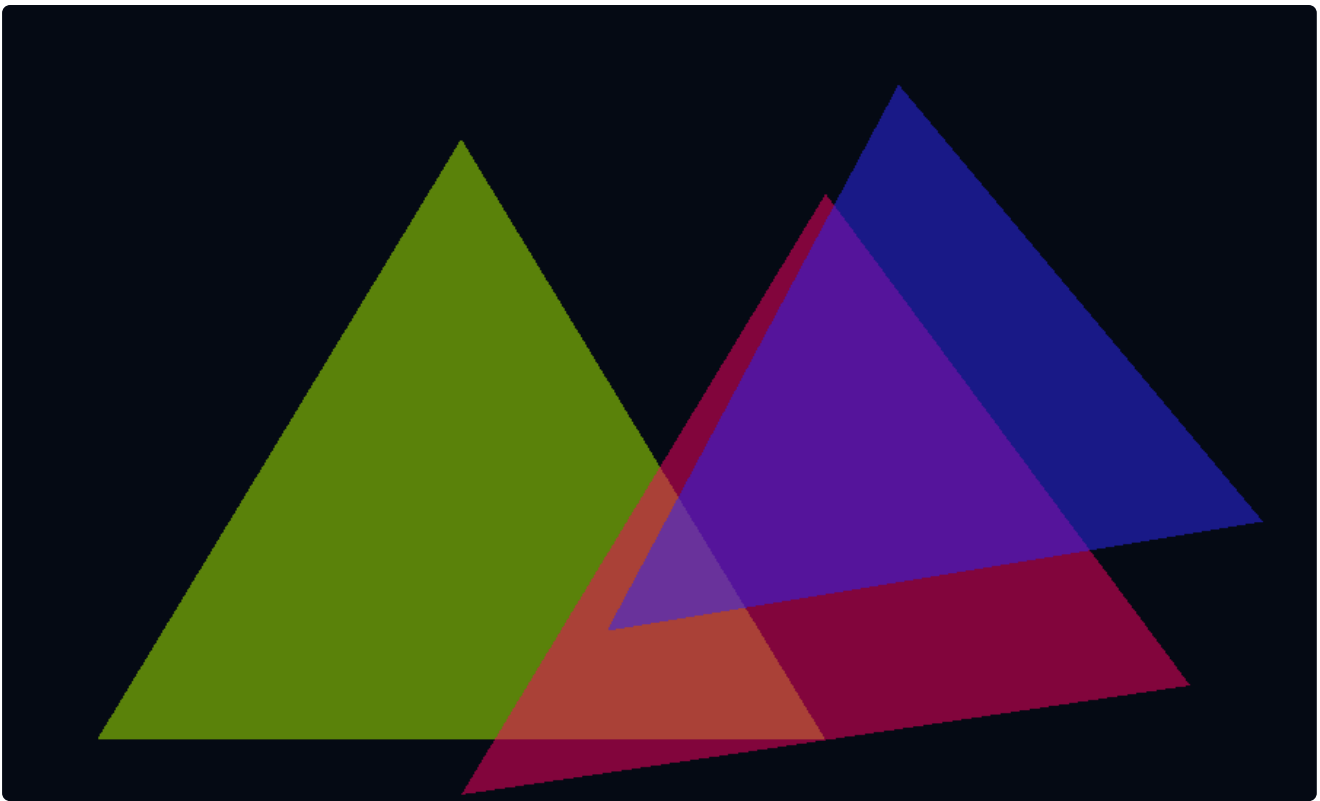
Briefly explain what OpenGL does in between the vertices for each fragment with the vertex attributes:

OpenGL first determines the triangle using the position attributes in the vertex, then it interpolates the color attribute for each fragment inside the triangle. The interpolation is why it creates a gradient when the vertex colors are unique.

Task 2: Alpha Blending and Depth

```
let vertices: Vec<f32> = vec![
    -0.8, -0.5, 0.0,  0.2, -0.5, 0.0, -0.3,  0.6, 0.0,
    -0.3, -0.6, 0.6,  0.7, -0.4, 0.6,  0.2,  0.5, 0.6,
    -0.1, -0.3, 0.3,  0.8, -0.1, 0.3,  0.3,  0.7, 0.3,
];
```

2a.) Draw three overlapping triangles with $a < 1$



Drew three triangles all with alpha set to 0.5.

2b.)

i.) Swap color of the triangles



Swapped the colors of the triangles. Because blending is enabled the color of the area where all three triangles are overlapping changes when the triangles are slightly transparent. In the overlapping spots the colors of the triangles are blended together, with the triangle with highest z having the most effect on the final color. When changing colors of the triangles, the spot where all the triangles overlap turned more green, while it originally was more purple.

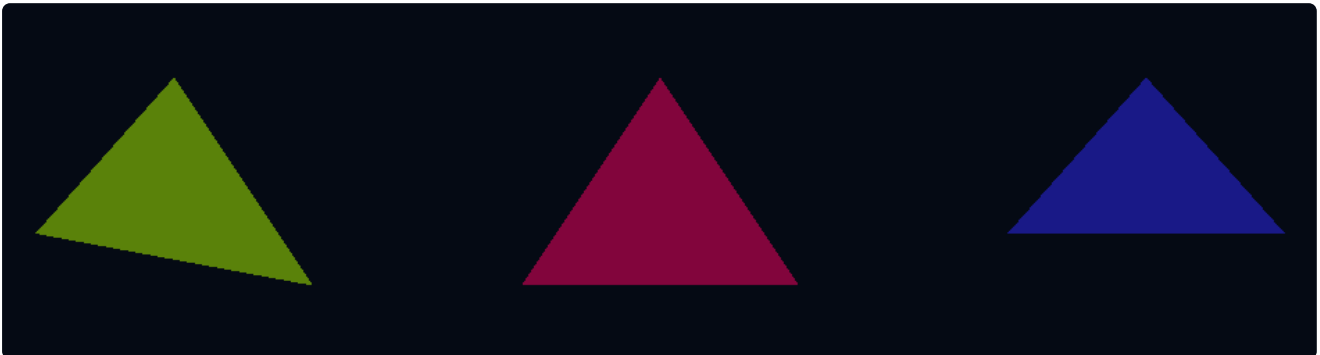
ii.) Change depth of triangles



Changed to colors back and swapped the z -coordinates of the triangles. The triangles were not drawn back-to-front anymore, but in the order of near $\rightarrow$ far $\rightarrow$ middle as green $\rightarrow$ pink $\rightarrow$ blue. In the areas with overlap with the green triangle, none of the other triangles contribute to the blending of the colors. When the near green triangle is drawn first, depth testing then tests the fragments in the other triangles and fail them as they are further back. This means they are discarded. But with the two other triangles, they are drawn in back-to-front order and hence we get blending of the pink and blue.

Task 3: The Affine Transformation Matrix

3a.)



...

The same vertices from 1b:

```
let vertices: Vec<f32> = vec![
    -0.9, -0.2, 0.0,  -0.5, -0.3, 0.0,  -0.7,  0.1, 0.0,
    -0.2, -0.3, 0.0,  0.2, -0.3, 0.0,   0.0,  0.1, 0.0,
    0.5, -0.2, 0.0,   0.9, -0.2, 0.0,   0.7,  0.1, 0.0,
];
```

And using this setup for the vertex shader:

```

layout(location = 1) in vec3 position;
layout(location = 2) in vec4 aColor;
out vec4 color;
mat4 matrix = mat4(
    1.0, 0.0, 0.0, 0.0,
    0.0, 1.0, 0.0, 0.0,
    0.0, 0.0, 1.0, 0.0,
    0.0, 0.0, 0.0, 1.0
);
uniform float time;
void main()
{
    matrix[0][0] = sin(time * 0.5);
    gl_Position = matrix * vec4(position, 1.0);
    color = aColor;
}

```

3b.) Determining different transformations

Changing a

Change: `matrix[0][0] = sin(time * 0.5);`

This creates the following transformation matrix

$$\begin{bmatrix} a & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This is a scaling transformation about the x-axis.

Changing b

Change: `matrix[1][0] = sin(time * 0.5);`

This creates the following transformation matrix

$$\begin{bmatrix} 1 & b & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This is a shear the x-axis, based on y.

Changing c

Change: `matrix[3][0] = sin(time * 0.5);`

This creates the following transformation matrix

$$\begin{bmatrix} 1 & 0 & 0 & c \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The transformation is translation about the x-axis.

Changing d

Change: `matrix[0][1] = sin(time * 0.5);`

This creates the following transformation matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ d & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The transformation is shearing the y-axis based on x.

Changing e

Change: `matrix[1][1] = sin(time * 0.5);`

This creates the following transformation matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & e & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The transformation is scaling about the y-axis.

Changing f

Change: `matrix[3][1] = sin(time * 0.5);`

This creates the following transformation matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & f \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The transformation is translation along the y-axis.

3c.) Why none of the transformations were rotations

Because rotation requires the rotation matrix which should have a specific combination of values. An arbitrary rotation will take the following form:

$$\begin{bmatrix} a & b & c & 0 \\ d & e & f & 0 \\ p & q & r & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

In this task only one value was changed at the time, at the most being the identity matrix and one other value. None of the changes created a matrix with the combination of any of the rotation matrices for any of the axes or an arbitrary rotation.

Task 4: Combination of Transformations

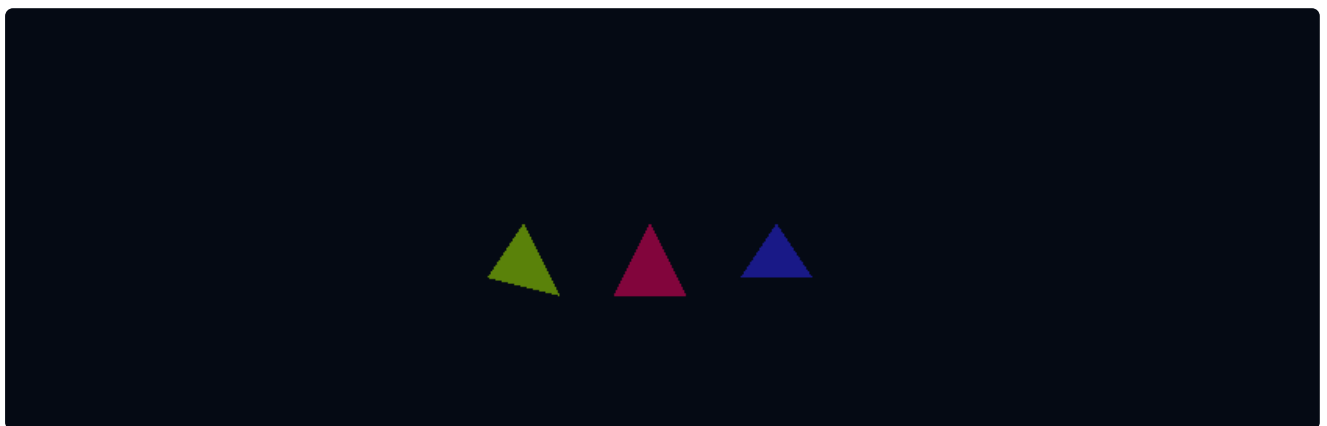
4a.) Uniform variable for the matrix

Changed the matrix in the vertex shader to a uniform variable by doing: `uniform mat4 matrix; .`

Then in `main.rs`, get the location of the variable outside the loop, defined a matrix and use them both to pass the matrix to the shader.

```
let matrix_location = unsafe {
    gl::GetUniformLocation(
        simple_shader.program_id,
        b"matrix\0".as_ptr() as *const i8)
};
// Then inside loop:
gl::UseProgram(simple_shader.program_id);
gl::UniformMatrix4fv(matrix_location, 1, FALSE, matrix.as_ptr());
```

4b.) Translation and perspective projection on triangles



Calculated transformations (translation and then the perspective projection) as the `matrix` sent to my shader:

```
let translation_matrix = glm::Mat4::new(
    1.0, 0.0, 0.0, 0.0,
    0.0, 1.0, 0.0, 0.0,
```

```
        0.0, 0.0, 1.0, -2.0,  
        0.0, 0.0, 0.0, 1.0  
    );
```

```
    let projection_matrix: glm::Mat4 =  
glm::perspective(window_aspect_ratio, 90.0, 1.0, 100.0);
```

```
    let matrix:glm::Mat4 = projection_matrix * translation_matrix;
```

4c.) Creating the camera

Keys used:

Key	Action
W	Up
A	Left
S	Down
D	Right
LShift	Zoom out
Space	Zoom in
Up	Pitch rotation up
Down	Pitch rotation down
Left	Yaw rotation left
Right	Yaw rotation right